HackerRank: Deep Learning Research Engineer Assignment

Part A: Understand and Reproduce the Baseline

A1. Understanding the Mamba-1 Implementation

Input propagation and state updates. In `Mamba.forward()`, event tokens are embedded and processed through residual Mamba blocks and final RMSNorm, followed by our five-class prediction head. Each `ResidualBlock` applies pre-normalization and residual addition. In `MambaBlock.forward()`, one projected branch passes through causal Conv1D, SiLU, and the selective SSM, while another provides output gating. The recurrent update in `selective_scan()` is

$$h_t = e^{\Delta_t A} \odot h_{t-1} + \Delta_t B_t u_t, \qquad y_t = C_t h_t + D u_t. \tag{1}$$

Thus, previous SET events can influence subsequent QUERY predictions.

Input-dependent parameters. In `ssm()`, `x_proj` produces `dt_rank + 2*d_state` features, split into a low-rank representation of Δ, B, and C. The `dt_proj` and Softplus operations produce positive Δ_t. Specifically, Δ_t controls previous-state decay and scales the new input contribution; B_t determines how the current input contributes to the recurrent state update; and C_t determines how the updated state contributes to the output. Unlike the shared learned parameters A and D, these quantities depend on the current input.

Distinct failure modes. Retention requires preserving a value across long SET-to-QUERY delays, where repeated multiplication by `deltaA` may attenuate stored information. Overwriting requires replacing an obsolete value, although `selective_scan()` adds `deltaB_u` rather than explicitly replacing state. Cross-key interference arises because different keys share the same recurrent representation without dedicated memory slots. Consequently, successful overwriting does not guarantee reliable long-delay retention.

Modification points and invariants. The relevant code paths are `MambaBlock.ssm()`, which constructs the selective parameters, and `MambaBlock.selective_scan()`, which computes `deltaA`, `deltaB_u`, and recurrent updates. Any modification must preserve causal processing, event order, compatible tensor shapes, and the surrounding MambaBlock interface.

A2. Task Head and Training Pipeline

Dataset and model. We use the provided synthetic generator with eight keys (A–H), four stored values (1–4), and UNSET class 0. Each SET, QUERY, or NOOP event is encoded as one of 41 logical tokens. The dataset contains 4,000 training sequences of length 32–128 and 500 sequences per evaluation split. In `EventDataset`, events are converted to token IDs, while labels are assigned only at QUERY positions. The original Mamba-1 backbone uses $d_{\text{model}} = 32$, $n_{\text{layer}} = 1$, $d_{\text{state}} = 8$, expand $= 2$, and $d_{\text{conv}} = 4$. We replace the vocabulary prediction head with a linear layer mapping the final hidden representations to five output classes.

Training and validation. The data loader pads variable-length sequences and assigns -100 to all non-QUERY target positions, including padding. We optimize query-only cross-entropy:

$$\mathcal{L} = -\frac{1}{|\mathcal{Q}|} \sum_{t \in \mathcal{Q}} \log p_\theta(y_t \mid x_{\le t}), \tag{2}$$

where $\mathcal{Q}$ denotes the QUERY positions. Training uses AdamW, learning rate 10^{-3}, batch size 16, and FP32 precision. Validation and test evaluation use batch size 4. The implementation preserves causal processing and excludes query answers from the input sequence.

A3. Establishing the Baseline

We initially trained the original Mamba-1 for 250 optimizer updates, obtaining 34.47% validation accuracy and 25.57% ID-128 accuracy, only slightly above the majority-class predictor (33.51% and 23.28%, respectively). To demonstrate meaningful learning, we extended the training budget to 2,000 updates, keeping the architecture and hyperparameters unchanged. We repeated training with three random seeds (42, 43, 44) and evaluated all checkpoints on identical test splits.

Table 1: Baseline query accuracy (%). Mamba results are mean ± standard deviation over three seeds. The majority-class predictor always predicts UNSET (0).

Split	Majority	Mamba-1	Oracle
Validation	33.51	87.02 ± 1.19	100
ID-128	23.28	83.72 ± 1.01	100
OOD-256	11.28	80.30 ± 1.20	100
OOD-512	5.97	78.15 ± 1.94	100
Delay stress-512	4.21	77.97 ± 1.55	100
Overwrite-512	4.28	78.20 ± 1.75	100

The exact non-neural oracle achieves 100% accuracy by replaying SET events and retrieving the latest stored value. Although Mamba-1 substantially outperforms the majority baseline, its accuracy declines as sequence length increases. More importantly, the designated long-gap probe achieves only $24.60 \pm 2.43\%$ accuracy, approximately the 25% chance level for four stored values, whereas the overwrite probe reaches $96.60 \pm 2.31\%$. This suggests that extreme long-delay retention is a more severe limitation than retrieving recent updates.

A4. Falsifiable Hypotheses

H1: Long-Horizon Retention. Retrieval accuracy decreases as the SET-to-QUERY delay increases because stored information may decay or become entangled with intervening updates.

H2: Decoupling State Decay and Input Updates. In Mamba-1, Δ_t controls both previous-state decay and the contribution of the current input. We hypothesize that independently controlling state decay through a learned retention gate improves long-delay retrieval while preserving the model's ability to incorporate new information.

Part B: Proposed Research Improvement

B1. Learned Retention Gate

In the original Mamba-1 recurrence, Δ_t controls both previous-state decay and the contribution of the current input:

$$h_t = e^{\Delta_t A} \odot h_{t-1} + \Delta_t B_t u_t. \tag{3}$$

This coupling motivates controlling state decay independently, without modifying how new information enters the recurrent state.

We introduce an input-dependent retention gate, $r_t = \sigma(W_r u_t + b_r)$, giving the modified recurrence:

$$\boxed{h_t = e^{r_t \Delta_t A} \odot h_{t-1} + \Delta_t B_t u_t.} \tag{4}$$

Here, $r_t \in (0, 1)$ is a scalar computed at each timestep and shared across the state dimensions. Since $A < 0$, smaller r_t reduces state decay and promotes retention. Importantly, the input contribution $\Delta_t B_t u_t$ remains unchanged. The proposed gate introduces additional input-dependent control over previous-state decay without altering the original input-update term.

B2. Implementation and Controlled Ablations

We implement the modification in `model_retention.py` by extending the original `MambaBlock` with `RetentionMambaBlock`. A linear projection `nn.Linear(64,1)`, followed by sigmoid, computes r_t from the current input representation. Only the state-decay computation `deltaA` in `selective_scan()` is modified; the input-update term and remaining Mamba-1 operations are preserved. The gate is initialized with zero weights and bias 3, giving $r_t \approx 0.953$ initially. This keeps the modified recurrence close to the original Mamba-1 dynamics ($r_t = 1$) at initialization, avoiding a large initial change in state decay. We compare four configurations:

- **Baseline:** Original Mamba-1 without the gate.
- **Learned gate:** Input-dependent r_t.
- **Disabled gate:** $r_t = 1$, recovering the original recurrence.
- **Fixed gate:** $r_t = 0.5$, testing constant decay reduction without input-dependent gating.

These controls test whether learned retention improves long-delay retrieval beyond the original and fixed-decay models. Failure to improve long-delay retrieval, or degradation of existing state-tracking capabilities, would challenge H2. All configurations use matched training budgets, seeds, and evaluation datasets.

B3. Correctness and Computational Overhead

Correctness. Automated tests verify oracle correctness, SET overwrites, NOOP handling, query-only loss masking, output shapes, and finite gradients. Causality is tested by changing future tokens and confirming that earlier predictions remain unchanged. We also verify that disabling the gate ($r_t = 1$) reproduces the original selective scan, while $r_t = 0.5$ changes its output. All tests passed.

Computational overhead. The learned gate increases the parameter count from 10,149 to 10,214 (+65, or 0.64%). On a Tesla T4 GPU, peak allocated memory increased from 43.21 MB to 44.23 MB. Across seeds 43 and 44, total training and evaluation runtime ranged from 227.72 to 246.19 seconds for the baseline and 230.87 to 237.81 seconds for the learned gate. These measurements indicate comparable computational costs, although runtime varies across Google Colab executions. The complete seed-level resource measurements are provided in the accompanying Colab notebook.

Part C: Controlled Experiments and Analysis

C1. Experimental Protocol

We evaluate the baseline, learned retention gate, and two controlled ablations using three random seeds $\{42, 43, 44\}$. All configurations are trained for 2,000 optimizer updates with AdamW, learning rate 10^{-3}, batch size 16, and FP32 precision. Evaluation uses batch size 4 and identical, fixed datasets across all runs.

Training sequences have lengths 32–128, while evaluation covers ID-128, OOD-256, OOD-512, and two length-512 stress tests targeting long-delay retention and overwriting. Each evaluation split contains 500 sequences.

We report query accuracy as mean ± standard deviation across seeds. To diagnose failure modes, accuracy is also stratified by SET-to-QUERY delay, overwrite count, and cross-key interference. The designated stress probes are evaluated separately from overall query accuracy.

C2. Overall Performance and Ablations

Table 2 compares the original Mamba-1, the learned retention gate, and two controlled ablations. All results are reported as mean ± standard deviation across three seeds.

Table 2: Query accuracy (%) across sequence lengths. Results are mean ± standard deviation over three seeds.

Model	ID-128	OOD-256	OOD-512
Baseline	83.72 ± 1.01	80.30 ± 1.20	78.15 ± 1.94
Learned gate	$\mathbf{89.63 \pm 0.69}$	$\mathbf{87.23 \pm 0.90}$	$\mathbf{85.43 \pm 0.60}$
Disabled ($r = 1$)	83.72 ± 1.01	80.30 ± 1.20	78.15 ± 1.94
Fixed ($r = 0.5$)	85.77 ± 1.73	83.26 ± 2.09	81.26 ± 1.80

As shown in Table 2 and Figure 1, the learned gate consistently outperforms the original Mamba-1, improving accuracy by 5.91, 6.93, and 7.28 percentage points at lengths 128, 256, and 512, respectively.

The learned gate consistently outperforms the original Mamba-1, improving accuracy by 5.91, 6.93, and 7.28 percentage points at lengths 128, 256, and 512, respectively. Its lower standard deviation at length 512 also indicates more consistent performance across seeds.

The disabled-gate ablation reproduces the baseline results, confirming that $r = 1$ recovers the original recurrence. The fixed gate ($r = 0.5$) improves OOD-512 accuracy to 81.26%, but remains below the learned gate (85.43%). This suggests that learned gating provides additional benefits over the tested constant decay reduction.

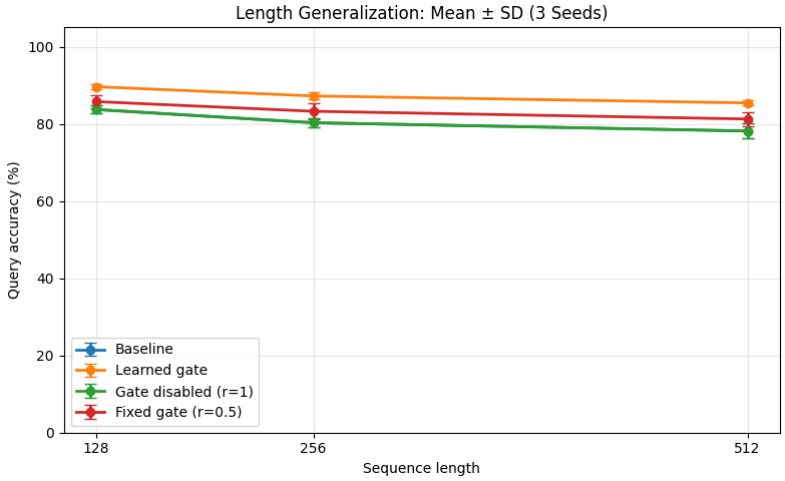


Figure 1: Query accuracy across sequence lengths with standard-deviation error bars over three seeds.

C3. Failure Mode Analysis

Long-horizon retention. We measure retention using the SET-to-QUERY delay, defined as the number of timesteps between a QUERY and the most recent SET operation for the same key. For each delay bin, query accuracy is the percentage of queries for which the model correctly predicts the latest stored value. Queries for keys that have never been assigned are excluded from this analysis.

Figure 2 shows that retrieval accuracy decreases as this delay increases, indicating difficulty in recovering information stored further in the past. On OOD-512, all models perform well at short delays but deteriorate beyond 64 timesteps. The learned gate consistently outperforms the baseline across the reported delay ranges, although substantial forgetting remains at longer delays.

Overwrite and cross-key interference. We measure overwrite count as the number of previous updates to the queried key beyond its initial assignment. Cross-key interference counts SET operations on other keys since the most recent update to the queried key. For each group, we compute the proportion of correctly answered queries.

Figure 3 shows that accuracy decreases after the first overwrite but remains relatively stable as overwrite counts increase further. In contrast, accuracy declines substantially with increasing cross-key interference, particularly beyond eight intervening updates. The learned gate maintains higher accuracy than the

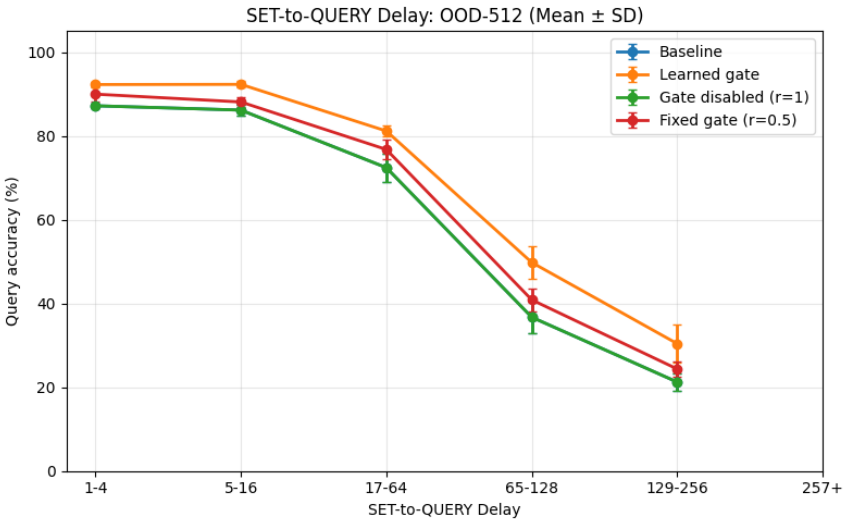


Figure 2: Query accuracy by SET-to-QUERY delay on OOD-512. Error bars show standard deviation across three seeds. The 257+ delay bin is omitted due to insufficient samples (fewer than 30 queries per seed).

baseline across both conditions, although interference-related degradation remains.

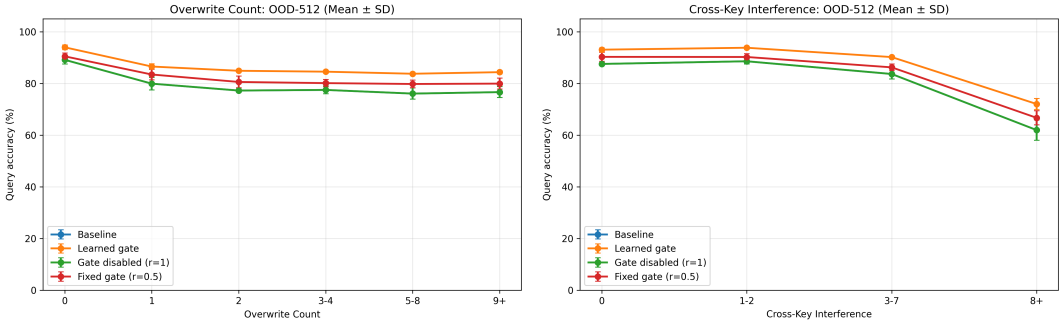


Figure 3: Query accuracy on OOD-512 by overwrite count (left) and cross-key interference (right). Error bars show standard deviation across three seeds.

Focused stress tests. We evaluate two targeted probes using the final QUERY in each length-512 sequence. The long-gap probe tests retrieval of a value stored at the first timestep, with a SET-to-QUERY delay of 511. The overwrite probe tests retrieval of the latest value after 1, 2, 4, or 8 overwrites of the same key.

Figure 4 reveals a pronounced difference between these conditions. On the long-gap probe, baseline accuracy is only $24.60 \pm 2.43\%$, while the learned gate achieves $22.87 \pm 2.20\%$, both near the 25% chance level for four stored values. This difference does not establish a reliable degradation. In contrast, overwrite-probe accuracy improves from $96.60 \pm 2.31\%$ to $99.53 \pm 0.12\%$.

These findings indicate that the learned gate improves general state tracking but does not resolve extreme long-gap retrieval. The remaining failures may involve interference from intervening updates or limitations in retrieving stored information, rather than state decay alone.

C4. Analysis of Learned Retention Gate

To examine the learned behavior, we measure the retention gate output r_t for SET, QUERY, and NOOP events using the first 20 sequences from the fixed ID-128 test set for each of the three trained seeds.

The gate learns values substantially below its initial value of 0.953 across all three seeds. Since $A < 0$, smaller r_t reduces the decay of the previous recurrent state. The observed values therefore indicate strong

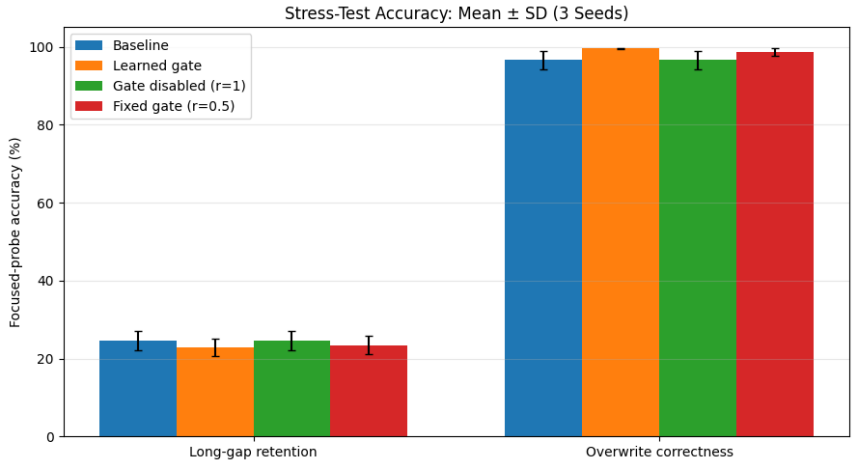


Figure 4: Focused long-gap retention and overwrite accuracy across three seeds (mean ± standard deviation).

Table 3: Mean learned retention gate values by event type.

Seed	SET	QUERY	NOOP
42	0.0389	0.0240	0.0323
43	0.0453	0.0136	0.0182
44	0.0542	0.0238	0.0286

suppression of state decay. The variation across event types also confirms that the gate is not constant.

However, despite this strong reduction in decay, extreme long-gap retrieval remains near chance level. Thus, reducing state decay alone is insufficient to guarantee reliable long-horizon retrieval. Furthermore, because the fixed-gate ablation uses $r = 0.5$, we cannot fully distinguish the benefit of input-dependent gating from that of learning a much smaller decay multiplier.

C5. Discussion, Limitations, and Next Experiment

Hypothesis assessment. H1 is supported, as retrieval accuracy declines with increasing SET-to-QUERY delay. H2 is partially supported: the learned gate improves OOD-512 accuracy by 7.28 percentage points, but extreme long-gap retrieval remains near chance (22.87% versus 24.60% for the baseline).

Primary bottleneck. Despite strong suppression of state decay, long-gap retrieval remains unreliable. This suggests that reducing decay alone is insufficient. Cross-key interference and retrieval limitations remain plausible explanations, but the experiments cannot conclusively distinguish them.

Limitations. Training on lengths 32–128 may limit generalization to 512-step sequences. Additionally, the learned gate produces values around 0.02–0.05, whereas the fixed-gate ablation uses $r = 0.5$. Comparing against a fixed $r = 0.03$ would help distinguish the benefit of input-dependent adaptation from that of stronger decay suppression.

Next experiment. We would compare long-gap retrieval under matched SET-to-QUERY delays with (i) intervening NOOP events and (ii) intervening SET operations on other keys. Using the same trained checkpoints would isolate the effect of distractor updates. A substantial performance gap would support cross-key interference as a contributor to long-horizon failure.